

Análisis de brechas y plan de mejoras en gestión de vulnerabilidades

Comparación del estado actual del sistema contra plataformas reales de gestión de vulnerabilidades (Qualys VMDR, Tenable/Nessus, Rapid7 InsightVM, OpenVAS/Greenbone, DefectDojo) y marcos de referencia (NIST SP 800-40, ISO 27001 Anexo A, PCI-DSS, CVSS/EPSS). Mejoras ordenadas de las más indispensables a las de menor prioridad.

Proyecto: GEF Secure — UTN Tesis

Fecha: 20/08/2026

Alcance: Backend Spring Boot · Frontend React · Pipeline n8n · PostgreSQL

1. Metodología

Este análisis parte del estado real del código de GEF Secure a la fecha (verificado directamente contra el repositorio, no supuesto), y lo compara contra:

- **Plataformas comerciales/open-source de referencia:** Qualys VMDR, Tenable.io/Nessus, Rapid7 InsightVM, OpenVAS/Greenbone, DefectDojo (este último el más cercano en filosofía a GEF Secure: orquesta datos de múltiples fuentes en vez de escanear él mismo).
- **Marcos y estándares:** NIST SP 800-40 (Guide to Enterprise Patch Management), ISO 27001 Anexo A.12 (Operations Security) y A.5.7 (Threat Intelligence), PCI-DSS Req. 6 y 11 (gestión de vulnerabilidades y SLA de remediación), CVSS v3.1/v4, y EPSS (Exploit Prediction Scoring System, mantenido por FIRST.org).

Cada ítem se clasifica en 4 niveles de prioridad y se estima su complejidad de implementación (S = días, M = 1-2 semanas, L = 3-4 semanas, XL = requiere una nueva capacidad de fondo).

2. Estado actual — lo que GEF Secure ya cubre bien

Antes de listar lo que falta, vale reconocer lo que ya está sólido, para que el análisis se lea en contexto:

- **Inventario de activos y componentes:** Entornos → Activos → Componentes de software, con importación automática de SBOM (Syft/CycloneDX).
- **Detección:** pipeline n8n contra GitHub Advisory Database (GHSA) y CISA KEV, con CVSS, prioridad y flag de zero-day/parche disponible.
- **Ciclo de vida de vulnerabilidades:** estado de detección (abierta/resuelta automático) separado del triage del analista, con reapertura trazada.
- **RBAC con 4 roles** (ADMIN, SECURITY_ANALYST, ASSET_OWNER, AUDITOR) y visibilidad acotada por activo asignado.
- **Tablero Kanban** con contexto rico por hallazgo (descripción real de la advisory, versión parcheada, links a NVD/GHSA).
- **Dashboard** con KPIs, MTTR real, tendencia 30 días, distribución por severidad/estado.
- **Historial de escaneos** con comparación entre dos puntos en el tiempo, paginado y filtrado.
- **Informes en PDF** (escaneo puntual, comparativo, resumen ejecutivo, ficha de CVE).
- **Notificaciones a Slack** ante hallazgos críticos.

3. **CRÍTICO** Indispensable

Estos ítems son los que cualquier evaluador de un sistema de gestión de vulnerabilidades va a buscar primero — sin ellos, el sistema no se puede considerar production-ready, aunque el resto funcione bien.

3.1 Autorización real del rol AUDITOR en el backend

Qué pasa hoy: el rol AUDITOR está pensado como solo-lectura, pero esa restricción **solo existe en el frontend** (Kanban.tsx deshabilita el drag-and-drop y los inputs). El backend no tiene ningún @PreAuthorize que mencione AUDITOR — cualquier usuario con ese rol puede llamar directamente

PATCH /api/vulnerabilities/{id}/status y modificar el estado de una vulnerabilidad, evadiendo la restricción por completo.

Por qué importa: esto es **Broken Access Control**, la categoría #1 del OWASP Top 10 2021. En un producto de ciberseguridad, un control de acceso roto en el propio producto es la peor clase de hallazgo — es exactamente lo que el sistema está diseñado para detectar en otros.

Cómo lo resuelven otras plataformas: todas (Qualys, Tenable, DefectDojo) fuerzan la autorización en la capa de API, nunca solo en la UI — la UI es una comodidad, no un control de seguridad.

Complejidad: S. Agregar `@PreAuthorize("hasAnyRole('ADMIN','SECURITY_ANALYST')")` a los endpoints de escritura de `VulnerabilityAuditController` (excluyendo `AUDITOR`), con un test que confirme el 403.

3.2 Estado de "riesgo aceptado" / "falso positivo" con justificación

Qué pasa hoy: triageStatus solo admite 3 valores (DETECTADA, EN_ANALISIS, RESUELTA). No existe forma de marcar una vulnerabilidad como falso positivo, o como riesgo aceptado formalmente (con justificación y fecha de revisión).

Por qué importa: en la práctica, no todo hallazgo se "resuelve" — muchos se aceptan (el parche rompe algo más importante, el activo se va a dar de baja pronto, el vector no es explotable en ese contexto) o son ruido (falsos positivos del scanner). Sin este estado, los analistas terminan marcando cosas como "RESUELTA" que no lo están, ensuciando las métricas (MTTR, tasa de resolución) y perdiendo la trazabilidad de la decisión real.

Cómo lo resuelven otras plataformas: Qualys, Tenable y DefectDojo tienen todos un estado explícito de "Risk Accepted"/"False Positive" con campo de justificación obligatorio y, en los más maduros, fecha de reevaluación automática.

Complejidad: M. Nuevos valores de triageStatus (RIESGO_ACEPTADO, FALSO_POSITIVO), campo justificacion + fecha_revision, ajuste de las transiciones permitidas y de las métricas del Dashboard para no contarlos como "resueltos".

3.3 SLA de remediación por severidad, con vencidos visibles

Qué pasa hoy: existe MTTR (tiempo *promedio* histórico), pero no hay una política de "esto debe resolverse en X días según su severidad" ni una vista de qué está vencido.

Por qué importa: es el requisito más citado en auditorías de seguridad y el corazón de PCI-DSS Req. 6.3.3/11.3.1 (plazos obligatorios de remediación por severidad). Sin esto, "gestión de vulnerabilidades" es solo "lista de vulnerabilidades" — falta la parte de gestión.

Cómo lo resuelven otras plataformas: todas permiten definir SLA por severidad (ej: Crítica 48h, Alta 7 días, Media 30 días, Baja 90 días) y calculan automáticamente qué está vencido, con alertas.

Complejidad: M. Config de SLA por severidad (tabla nueva o valores fijos en config), columna calculada "vencido/días restantes" en AssetVulnerability, badge visual en el Kanban, filtro "vencidos" y una quinta sección en Informes.

3.4 Criticidad del activo / contexto de exposición

Qué pasa hoy: Asset/AssetType no tienen ningún campo de criticidad de negocio, exposición a internet, o sensibilidad del entorno. Todas las vulnerabilidades se priorizan igual sin importar en qué corren.

Por qué importa: el riesgo real no es solo severidad — es **severidad × criticidad del activo × exposición**. Una CVE crítica en un servidor de pruebas interno no es lo mismo que la misma CVE en un servidor de producción expuesto a internet. Sin este eje, el Kanban trata ambos casos igual, lo que en la práctica hace que el equipo pierda tiempo en lo que no importa y posiblemente ignore lo que sí.

Cómo lo resuelven otras plataformas: Qualys y Tenable calculan un "Asset Risk Score" combinando la criticidad declarada del activo con la severidad de sus hallazgos; es una de las funciones más vendidas de esas plataformas.

Complejidad: M. Campo criticidad (ALTA/MEDIA/BAJA) y expuestoInternet (bool) en Asset, ordenamiento del Kanban/Dashboard por un score combinado en vez de solo CVSS.

3.5 Trail de auditoría (quién cambió qué, cuándo, de qué a qué)

Qué pasa hoy: AssetVulnerability guarda el estado *actual* (asignado a quién, decisión, fecha de resolución) pero no un historial de cambios. Cuando alguien reasigna o cambia el estado, se pisa el valor anterior sin dejar rastro.

Por qué importa: el sistema tiene un rol **AUDITOR** dedicado, pero hoy no hay nada específico que auditar más allá del estado actual — que es exactamente lo que ese rol necesitaría ver. Es además un requisito de ISO 27001 A.12.4 (logging) y de cualquier evaluación de madurez en gestión de vulnerabilidades.

Cómo lo resuelven otras plataformas: todas mantienen un log de eventos por hallazgo (change history / activity feed) visible en el propio detalle del hallazgo.

Complejidad: M/L. Tabla vulnerability_status_history (o event sourcing liviano), poblada en cada updateStatus(), con una pestaña "Historial" en el modal del Kanban y una vista dedicada para el rol AUDITOR.

3.6 Hardening de autenticación

Qué pasa hoy: el login es usuario/contraseña con BCrypt, sin bloqueo por intentos fallidos, sin rate limiting en /api/auth/login, sin políticas de complejidad de contraseña, y sin segundo factor (MFA/2FA).

Por qué importa: en un producto de ciberseguridad esto es doblemente sensible — comprometer una cuenta acá no solo afecta datos, da acceso a la visión completa de las vulnerabilidades de la organización (un mapa de ruta para un atacante). Es de los primeros puntos que revisa cualquier pentest.

Cómo lo resuelven otras plataformas: MFA (TOTP) es estándar en todas las plataformas comerciales de este rubro; bloqueo tras N intentos fallidos y rate limiting son básicos en cualquier login moderno.

Complejidad: M (rate limiting + bloqueo) a L (MFA con TOTP).

3.7 HTTPS/TLS y cabeceras de seguridad

Qué pasa hoy: nginx.conf sirve el frontend solo por HTTP (puerto 80), sin bloque TLS, y sin cabeceras de seguridad (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) ni en nginx ni en Spring Security.

Por qué importa: sin HTTPS, las credenciales (usuario/contraseña, JWT) viajan en texto plano en la red — cualquiera en el mismo segmento de red puede capturarlas. Es un mínimo indiscutible para cualquier despliegue fuera de localhost, más aún en una herramienta de seguridad.

Cómo lo resuelven otras plataformas: TLS obligatorio (con redirect 80→443 automático) y cabeceras de seguridad completas son la base de cualquier producto SaaS de seguridad.

Complejidad: S/M. Certificado (Let's Encrypt vía Certbot, o terminación TLS en el proveedor de hosting), bloque `server { listen 443 ssl; }`, cabeceras vía `add_header` en nginx.

4. **ALTA** Alta prioridad

Importantes para la madurez del producto, pero el sistema es funcional sin ellos.

4.1 EPSS (Exploit Prediction Scoring System)

Qué falta: hoy solo se prioriza por CVSS (severidad teórica). No se usa EPSS, que estima la **probabilidad real de que ese CVE sea explotado en los próximos 30 días** — es el complemento moderno estándar a CVSS, y CISA lo recomienda activamente desde 2023 para priorizar remediación.

Cómo lo resuelven otras: Tenable y Rapid7 ya combinan CVSS + EPSS para el ranking de prioridad por defecto.

Complejidad: M. La API de EPSS (`api.first.org/data/v1/epss`) es pública y gratuita, mismo patrón que ya se usa para GHSA — se puede cachear igual que las advisories.

4.2 Vector CVSS completo (no solo el número)

Qué falta: se guarda `cvss` como un string numérico ("7.5"), sin el vector completo (`AV:N/AC:L/PR:N/UI:N/...`). Sin el vector, el analista no puede saber *por qué* algo es crítico (¿es explotable remotamente sin autenticación, o requiere acceso local?).

Complejidad: S. El vector ya viene en la respuesta de GHSA (`cvss_v3.vector_string`, visto en el propio pipeline de n8n) — es exponerlo, no obtenerlo.

4.3 Retención y archivado de datos históricos

Qué falta: `scan_reports` y `vulnerabilities_audit` crecen sin límite — no hay ninguna tarea de limpieza o archivado.

Por qué importa: con escaneos frecuentes, estas tablas crecen indefinidamente, degradando performance de consultas y backups con el tiempo. Toda plataforma real define una política de retención (ej: detalle completo 12 meses, agregados después).

Complejidad: M. Job programado (`@Scheduled` en Spring) que archive o resuma filas más viejas que N meses.

4.4 Programación de escaneos configurable desde la app

Qué falta: el `ScanScheduler` de n8n corre con un horario fijo (diario, hardcodeado en el JSON del workflow). No hay forma de cambiar la frecuencia por entorno o activo sin editar el pipeline directamente en n8n.

Por qué importa: un administrador no debería necesitar tocar n8n para ajustar cada cuánto se escanea un entorno crítico vs. uno de bajo riesgo.

Complejidad: M. Tabla de configuración de frecuencia por entorno + que n8n la consulte al backend antes de decidir qué escanear en cada corrida programada.

4.5 Notificaciones por email

Qué falta: el único canal de notificación es Slack. No hay email ni ningún otro canal.

Por qué importa: no todos los interesados (gerencia, auditoría externa, un dueño de activo que no usa Slack) están en el mismo espacio de Slack de la organización — email sigue siendo el canal más universal para reportes periódicos y alertas críticas.

Complejidad: S/M. Nodo SMTP en n8n (o `spring-boot-starter-mail` del lado Java), reutilizando el mismo contenido que ya arma la sección Informes.

4.6 Alcance de escaneo: reconocer explícitamente el límite actual

Qué falta: GEF Secure hoy es **SCA puro** (Software Composition Analysis vía SBOM) — no escanea sistema operativo, puertos abiertos, configuración de red, ni TLS/certificados, que es donde Nessus/OpenVAS/Qualys ponen la mayor parte de su cobertura.

Por qué importa: no es necesariamente algo para "arreglar" en el corto plazo (es una decisión de alcance de tesis válida), pero es importante documentarlo explícitamente como limitación conocida, no dejarlo implícito — cualquier comparación seria con esas plataformas lo va a señalar primero.

Complejidad: XL si se decide cubrir (requeriría integrar un scanner de red tipo OpenVAS/Nmap+Nikto). Se lista acá para que la decisión de dejarlo fuera de alcance sea explícita y consciente, no un olvido.

5. **MEDIA** Prioridad media

Mejoran la madurez del producto una vez resuelto lo anterior, no son bloqueantes.

- **5.1 Watchlist / alerta proactiva de CVEs nuevos.** Hoy solo se detectan CVEs en el próximo escaneo programado. Una organización con software crítico querría enterarse el mismo día que se publica un CVE que la afecta, no esperar al próximo ciclo. Requiere un proceso separado que cruce el catálogo de software inventariado contra nuevas advisories publicadas (polling diario contra GHSA). *Complejidad: M.*
- **5.2 Mapeo a marcos de cumplimiento** (ISO 27001, PCI-DSS, NIST CSF). Etiquetar hallazgos contra controles de un marco específico, para que un reporte de auditoría se arme filtrando por control en vez de por severidad. Reconocido ya como fuera de alcance del sistema de Informes actual — se reafirma acá como mejora futura, no urgente. *Complejidad: L.*
- **5.3 Integración con sistemas de tickets** (Jira/ServiceNow). Muchas organizaciones no gestionan la remediación en el Kanban de la herramienta de seguridad, sino en su sistema de tickets de IT ya existente. Sin esta integración, el Kanban es una isla que el equipo de infraestructura no necesariamente visita. *Complejidad: L.*
- **5.4 Visualización de dependencia transitiva.** Saber si un paquete vulnerable es una dependencia directa o heredada de otra ayuda a decidir la vía de remediación real. Syft/CycloneDX ya captura esta relación en el SBOM — hoy se descarta al importar, solo se guarda la lista plana. *Complejidad: M (el dato ya está disponible, falta modelarlo y mostrarlo).*

- **5.5 Reporte dedicado de cumplimiento de SLA.** Una vez exista el punto 3.3, un quinto tipo de informe en la sección Informes ("¿qué está vencido, por cuánto, y quién es responsable?") cierra el círculo de ese esfuerzo. *Complejidad: S, una vez resuelto 3.3.*

6. **BAJA** Evolución futura

Válidas como roadmap de largo plazo, pero no urgentes para el alcance actual del proyecto.

- **Escaneo de imágenes de contenedor** (capas Docker, CVEs del sistema operativo base) — disciplina relacionada pero distinta (container security), común en plataformas modernas (Trivy, Qualys Container Security) pero fuera del foco actual de SCA sobre componentes de software.
- **SAST** (análisis estático de código propio) — otra disciplina de AppSec, complementaria pero independiente de la gestión de vulnerabilidades de terceros que hace GEF Secure hoy.
- **Detección de secretos hardcoded** (estilo gitleaks/trufflehog) — típico complemento moderno de SCA, bajo impacto si el proceso de desarrollo ya usa un gestor de secretos.
- **Multi-organización / multi-tenant** — solo relevante si GEF Secure fuera a ofrecerse a más de una organización cliente; para uso interno de una sola empresa, no aporta hoy.
- **API pública con API keys** para integraciones externas — bajo prioridad mientras el consumo sea solo el frontend propio.

7. Tabla resumen

#	Mejora	Prioridad	Complejidad
3.1	Autorización real del rol AUDITOR en el backend	Crítico	S
3.2	Riesgo aceptado / falso positivo con justificación	Crítico	M
3.3	SLA de remediación por severidad	Crítico	M
3.4	Criticidad de activo / exposición	Crítico	M
3.5	Trail de auditoría (historial de cambios)	Crítico	M/L
3.6	Hardening de autenticación (rate limit, bloqueo, MFA)	Crítico	M–L
3.7	HTTPS/TLS + cabeceras de seguridad	Crítico	S/M
4.1	EPSS junto a CVSS	Alta	M
4.2	Vector CVSS completo	Alta	S
4.3	Retención/archivado de datos históricos	Alta	M
4.4	Programación de escaneos configurable	Alta	M
4.5	Notificaciones por email	Alta	S/M
4.6	Documentar el alcance SCA-only explícitamente	Alta	— (decisión)
5.1	Watchlist proactivo de CVEs nuevos	Media	M
5.2	Mapeo a marcos de cumplimiento	Media	L

#	Mejora	Prioridad	Complejidad
5.3	Integración con Jira/ServiceNow	Media	L
5.4	Dependencias transitivas	Media	M
5.5	Reporte de cumplimiento de SLA	Media	S*
6.x	Contenedores / SAST / secretos / multi-tenant / API pública	Baja	L-XL

Documento generado como cierre de la sesión del 19-20/08/2026 — ver Cambios_Sesion_Escaneos_Inventario.md para el detalle de lo implementado en esta misma sesión. GEF Secure — UTN Tesis.